

Тема 1 Взаимодействие программ с командной оболочкой и средой окружения

1.1. Взаимодействие программ с командной оболочкой (строкой запуска)

В программе, написанной на языке Си, первой выполняемой функцией является функция **main**. Традиционно функция `main()` определяется следующим образом:

```
int main(int argc, char *argv[], char *envp[]) {
```

Из заголовка функции видно, что она имеет три параметра и возвращает целое значение. Первым параметром (**argc**) в функцию передаётся число аргументов, указанных в командной строке при запуске программы, включая её имя. Указатели на каждый из аргументов командной строки передаются в массиве **argv[]**. Таким образом, `argv[0]` указывает на строку, содержащую имя программы, `argv[1]` – на первый аргумент и т.д. до `argv[argc – 1]`. Элемент `argv[argc]` всегда содержит NULL.

Массив `envp[]` содержит указатели на переменные окружения, передаваемые программе. Каждая переменная представляет собой строку вида

имя_переменной = значение_переменной

Возвращаемое значение функции *main* является статусом завершения программы (процесса).

Приведём пример программы, которая выводит значения всех аргументов, переданных функции *main* (листинг 1).

Листинг 1. Вывод информации о командной строке

```
#include <stdio.h>
int main(int argc, char *argv[]){
    int i;
    printf ("Имя программы - %s\n", argv[0]);
    printf ("Число параметров - %d\n", argc-1);
    for (i = 1; i < argc; i++)
        printf("argv[%d] = %s\n", i, argv[i]);
    return (0);
}
```

Чтобы определить, были ли указаны в командной строке опции, аргументы командной строки могут быть обработаны при помощи следующих функций:

```

#include <unistd.h>
#include <getopt.h>
extern char *optarg;
extern int optind, opterr, optopt;

int getopt(int argc, char *argv[], char
*optstring);

int getopt_long(int argc, char *argv[],
                char *optstring,
                struct option *longopts,
                int *longindex);

int getopt_long_only(int argc, char *argv[],
                    char *optstring,
                    struct option *longopts,
                    int *longindex);

```

Напомним, что *опцией* называется аргумент командной строки, начинающийся с символов «-»(тире) или «--» (двойное тире). Причём, если используется один знак тире, то опция считается односимвольной или *короткой*, если же два знака тире, то многосимвольной или *длинной*.

Функция **getopt** просматривает аргументы командной строки по очереди и определяет, есть ли в них заданные короткие опции. Поиск только длинных опций осуществляется функцией **getopt_long_only**. Функция **getopt_long** осуществляет поиск как коротких, так и длинных опций.

Поиск коротких опций функцией *getopt* производится так же, как командой *getopts*, используемой в скриптах (см. часть 1 курса). Отличия следующие.

1. Если просмотрены все параметры командной строки, то *getopt* возвращает -1.

2. Если в *optstring* после некоторого символа следует два двоеточия, то это означает, что опция имеет **необязательный** аргумент. Это является дополнением GNU.

3. Если *optstring* содержит символ «W», за которым следует точка с запятой («;»), то опция -W *foo* рассматривается как длинная опция --*foo*. Опция -W зарезервирована POSIX.2 для реализации расширений. Такое поведение является дополнительным для GNU и недоступным в библиотеках GNU libc более ранних по сравнению со второй версий.

По умолчанию *getopt* переставляет элементы содержимого *argv* в процессе поиска так, что, в конечном счёте, все аргументы, не являющиеся

опциями, оказываются в конце. Возможны два других режима работы функции.

4. Если первым символом *optstring* является «+» или задана переменная окружения **POSIXLY_CORRECT**, то обработка опций прерывается на первом аргументе, не являющемся опцией.

5. Если первым символом *optstring* является «-»(тире), то каждый элемент *argv*, не являющийся опцией, обрабатывается так, как если бы он был аргументом опции с символом, имеющим код 1 (один). Такой режим работы используется программами, которые требуют опции и другие аргументы командной строки.

6. Специальный аргумент «--» (два тире) служит для обозначения конца опций независимо от режима работы функции *getopt*.

Если *getopt* не распознал символ опции, то он выводит в стандартный поток ошибок соответствующее сообщение, заносит символ в переменную *optopt* и возвращает «?». Вызывающая программа может предотвратить вывод сообщения об ошибке, установив нулевое значение **opterr**.

Пример программы, анализирующей опции командной строки, представлен в листинге 2.

Листинг 2. Работа с опциями командной строки и переменными среды окружения

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <getopt.h>
4.
5. int main (int argc, char **argv) {
6.     int c;
7.     while (1) {
8.
9.         c = getopt(argc, argv, "abc:d:012");
10.
11.         if (c == -1) break;
12.         switch (c) {
13.             case '0':      printf ("опция 0\n");
14.                             break;
15.             case '1':      printf ("опция 1\n");
16.                             break;
17.             case '2 ':     printf ("опция 2\n");
18.                             break;
19.             case 'a':      printf ("опция a\n");
20.                             break;
21.             case 'b':      printf ("опция b\n");
22.                             break;
23.             case 'c':      printf ("опция c со значением
24.                                 '%s'\n", optarg);
```

Листинг 2. Работа с опциями командной строки и переменными среды окружения

```
22.             break;
23.  case 'd': printf ("опция d со значением '%s'\n",
    optarg);
24.             break;
25.  case '?': break;
26.  default: printf ("результат 0%o ??\n", c);
27.  }
28.  }
29.  if (optind < argc){
30.  printf ("элементы ARGV, не параметры: ");
31.  while (optind < argc)
32.  printf ("%s ", argv[optind++]);
33.  printf ("\n");
34.  }
35.  exit (0);
36. }
```

2. Взаимодействие Си-программы со средой окружения

Массив **envp[]**, передаваемый третьим аргументом в функцию *main*, содержит указатели на переменные среды окружения. Каждый элемент массива является указателем на строку вида: «имя_переменной = значение». Последним элементом массива является указатель NULL.

Стандарт ANSI C определяет только два первых параметра функции *main* – *argc* и *argv*. Стандарт POSIX.1 определяет также параметр *envp*, хотя рекомендует передачу окружения программы производить через глобальную переменную **environ**:

```
extern char ** environ;
```

Формат массива *environ* такой же, как и у *env*. Пример вывода переменных среды окружения с использованием двух массивов приведён в листинге 3.

Листинг 3. Вывод информации о командной строке

```
#include <stddef.h>
#include <stdio.h>
extern char **environ;
int main(int argc, char *argv[]){
    int i;
    printf ("Запущена программа %s. ", argv[0]);
    printf ("Число параметров %d\n", argc-1);
    for (i = 1; i < argc; i ++){
        printf("argv[%d] = %s\n", i, argv[i]);
    }
    for (i = 0; i < 8; i ++)
```

Листинг 3. Вывод информации о командной строке

```
    if (environ[i] != NULL)
        printf("environ[%d] : %s\n", i, environ[i]);
}
```

Для доступа к переменным окружения по их именам используются функции: **getenv** и **putenv**:

```
#include <stdlib.h>

char *getenv(const char *name);
int putenv (const char *string);
```

Функция *getenv* возвращает значение переменной окружения с именем *name*, а *putenv* помещает переменную и её значение в окружение процесса. Параметр *string* функции *putenv* содержит строку вида *var_name = var_value*.

В качестве примера приведём программу, похожую по своей функциональности на предыдущую, выводящую выборочно значения переменных и устанавливающую новые значения по желанию пользователя.

Листинг 4. Работа с переменными среды окружения

```
1 #include <stddef.h>
2 #include <stdlib.h>
3 #include <stdio.h>
4
5 int main(int argc, char *argv[]){
6     char *term;
7     char buf[200], var[200];
8
9     /* Проверим, определена ли переменная TERM */
10    if ((term = getenv("TERM")) == NULL){
11        /* Если переменная не определена, получим от *
12         * пользователя её значение и поместим      *
13         * переменную в окружение программы        */
14        printf ("Переменная TERM не определена.\n");
15        printf ("Введите её значение: ");
16        gets(buf);
17        sprintf(var, "TERM=%s", buf);
18        putenv(var);
19        printf("%s\n", var);
20    }
21    else {
22        /* Если переменная TERM определена, предоставим *
23         * пользователю возможность изменить её значение, *
24         * после чего поместим её в окружение процесса */
25        printf("TERM=%s. Change? [N]", term);
```

Листинг 4. Работа с переменными среды окружения

```
2   gets(buf);
2   if (buf[0] == 'Y' || buf[0] == 'y') {
2       printf("TERM="); gets(buf);
2       sprintf(var, "TERM=%s", buf); putenv(var);
2       printf("new %s\n", var);
2   }
2 }
2 }
```

В строке 11 проверяется, определена ли переменная *TERM*. Если переменная *TERM* не определена, функция *getenv* возвращает NULL.

В строках 17 и 29 формируются аргументы для функции *putenv*, содержащие новые значения переменной *TERM*. Далее, в строках 18 и 29 соответственно, устанавливается новое значение для переменной *TERM*.

Введённое значение переменной будет действительно только для данного процесса и порождённых им процессов: если после завершения программы вывести значение *TERM*, то видно, что оно не изменилось.

Практическая работа №2.

Взаимодействие программ с командной оболочкой и средой окружения

Задание на лабораторную работу

Написать программу на языке Си, состоящую из двух файлов: заголовочного файла и файла программы.

В заголовочном файле должны быть подключены необходимые системные заголовочные файлы, и описаны все глобальные переменные и функции, используемые в файле программы.

Программа должна выполнять следующие действия:

- 1) вывести первые 10 переменных окружения на экран двумя различными способами (способ задаётся пользователем в командной строке опциями -1 и -2);
- 2) обработать все указанные опции командной строки и в случае ошибочных опций выдать сообщение в стандартный поток ошибок о неправильном использовании программы;
- 3) в командной строке опцией *-f файл* указывается имя файла, который необходимо открыть и вывести его содержимое на экран. Также необходимо обработать ошибки, которые могут возникнуть при открытии, закрытии и выводе файла на экран;
- 4) при завершении программы (даже в случае возникновения ошибок) на экран должно быть выдано сообщение об авторе программы: ФИО, идентификатор пользователя.

Контрольные вопросы

1. Способы получения переменных окружения.
2. Обработка параметров командной строки без использования системных функций.
3. Функции *getopt* и *getopt_long*. Принципы их работы.